# Telco Customer Churn

Early Warning System — Full Project Plan

## Plan A: Current Plan — Identified Gaps

The following table maps every gap in the original proposal to its associated risk level. These must all be addressed in the final implementation.

| Area | Gap | Risk |
|---|---|---|
| Imbalance handling | No method specified (SMOTE? Weighting? Threshold tuning?) | Model silently favours majority class |
| Validation strategy | K-fold mentioned but not integrated with model selection | Results not reproducible or trustworthy |
| Baseline | No dummy classifier baseline (e.g. always predict majority) | No performance floor to compare against |
| Target leakage | Churn Score & Churn Reason flagged but not proven excluded in code | Critical failure if included |
| Feature selection | AIC + Chi-squared mentioned but no decision rule stated | Arbitrary feature dropping |
| SHAP values | Listed as 'if time allows' | Weak interpretability commitment |
| Business framing | No cost-benefit framing (FN vs FP cost) | Recommendations feel generic |
| Hyperparameter tuning | Not mentioned at all | Models left at defaults |
| Dashboard | Listed as optional extension | Unlikely to be executed |

## Plan B: Strongest Possible Execution Plan

### Step 1 — Frame the Business Problem First

- Define the cost asymmetry explicitly before writing any code.
- False negative (missed churner) = lost customer lifetime value.
- False positive (unnecessary retention offer) = wasted discount cost.
- This justifies your metric choices and threshold decisions to a grader. Write this up as a short paragraph at the top of your notebook.

### Step 2 — Data Preprocessing

- Drop leakage columns first and document why each is dropped.
- Convert Total Charges to numeric; document the ~11 blank rows and your imputation decision.
- One-hot encode categoricals; check for multicollinearity with a correlation heatmap.
- Normalize continuous features for logistic regression only — not tree models.
- Document every decision. Graders reward transparency.

### Step 3 — Establish a Dummy Baseline

- Run DummyClassifier(strategy='most_frequent') before any real model.
- This sets your performance floor. Every subsequent model must beat it meaningfully.
- Report its confusion matrix, precision, recall, and F1 alongside the real models.

### Step 4 — Handle Class Imbalance Explicitly

- Test two approaches and compare results:
- (a) class_weight='balanced' on all three models.
- (b) SMOTE applied to training data only — never the test set (common mistake).
- Report which approach improves recall without collapsing precision.

### Step 5 — Feature Selection (Rigorous)

- Chi-squared test for all categorical predictors vs. churn label.
- Point-biserial correlation for continuous variables.
- VIF (Variance Inflation Factor) to detect and remove multicollinear features.
- Document every dropped feature and the threshold/rationale used.

### Step 6 — Train Models with Cross-Validation

- Use StratifiedKFold(n_splits=5) — stratified because of class imbalance.
- Logistic Regression with class_weight='balanced'.
- Random Forest: tune n_estimators, max_depth via GridSearchCV.
- Gradient Boosting: tune learning_rate, n_estimators, max_depth.
- Tune hyperparameters on training folds only. Touch the test set only once — for final evaluation.

### Step 7 — Evaluate Properly

- Confusion matrix for all three models.
- Precision, Recall, F1 for the churn class specifically.
- AUC-ROC with plotted curves overlaid for all three models.
- Precision-Recall curve — more informative than ROC under class imbalance.
- Training vs. validation performance gap (explicit overfitting check).

### Step 8 — Interpret Results (This Separates Good from Great)

- Logistic regression: coefficients with confidence intervals.
- Tree models: feature importance bar plots.
- SHAP values for at least one model — do not skip this. It is the difference between a good project and a great one.
- Translate top 5 features into plain business language below each plot.

■ Most teams stop at reporting metrics. Graders notice when you connect model output back to the business problem.

### Step 9 — Write Business Recommendations

- Write 3–5 specific, defensible recommendations based on your findings.
- Example: 'Customers on month-to-month contracts with tenure under 12 months and monthly charges above $70 represent X% of churners. Offering a discounted annual contract at month 6 could retain this segment.'
- Quantify where possible. Vague recommendations are penalized.

### Step 10 — Document Everything

- Comment your code throughout — not just at function level.
- Write a short Limitations section: dataset age, no deployment context, static model.
- Acknowledge what you would do differently with more time or data.
- This signals intellectual maturity and pre-empts grader criticism.

---

Note: No plan guarantees a perfect grade — execution, code quality, and grader judgment all matter. This plan eliminates every identifiable gap and gives you the strongest possible foundation.